

Bilinear by Default:

Koopman Rollouts for Sampling-Based Predictive Control

Abstract—The rollout model is the central design choice in sampling-based predictive control: Model Predictive Path Integral (MPPI) control is derivative-free and massively parallel, but only as good as the model through which it rolls out trajectories. Koopman operator models are attractive here, lifting nonlinear dynamics to an almost-linear evolution whose rollouts are inexpensive matrix–vector products with data-driven error bounds. Existing Koopman-MPPI keeps the lifted model *linear in the input*, inheriting a restriction that convex Koopman MPC needs but sampling-based control does not. This paper argues it should simply be dropped: for control-affine systems the provably correct *bilinear* Koopman model is the right default rollout, usable at the same matrix–vector cost precisely because sampling imposes no convexity. Keeping the lift linear is then not a neutral simplification but a structural forfeiture: it averages the state-dependent input map into a constant, erasing the input channel exactly when the input couples to the tracked motion through a configuration-dependent map such as a vehicle’s heading or a manipulator’s Jacobian. An online error-tube cost with a free-energy practical-stability guarantee makes the rollout error-aware, while a first- versus second-order criterion delimits the narrow regime where a linear lift still suffices. Across MuJoCo benchmarks it parks a nonholonomic vehicle in 95% of trials across seeds, matching a true-dynamics oracle, where linear deep-Koopman MPPI structurally fails (0%); on a manipulator the linear model reaches no target on any seed, whereas the bilinear lift, when its high-dimensional fit converges, reaches them at oracle accuracy, and it needs a four times smaller lifting network where both succeed. Full hyperparameters and supplementary videos are on the project page: https://koopman_mppi_anonymous.github.io.

Index Terms—Model predictive path integral control, Koopman operator, bilinear systems, sampling-based MPC, optimization and optimal control, robot learning.

I. INTRODUCTION

SAMPLING-BASED model predictive control, most prominently Model Predictive Path Integral (MPPI) control [1]–[3], has become a workhorse for agile robotics. It is derivative-free, embarrassingly parallel on GPUs, and, through its information-theoretic derivation [4], [5], places essentially no structural requirement on the dynamics it rolls out. This generality comes at a price that is easy to overlook: at every control step the controller propagates thousands of sampled trajectories over a horizon, so the *rollout model* is both the dominant computational cost and the single most important design choice. A high-fidelity physics engine or neural network gives accurate rollouts but is slow; a crude analytical model is fast but biases the control. The quality of MPPI is, to first order, the quality of this speed–accuracy trade-off [6].

The Koopman operator offers a principled way to make the trade favorable. By lifting the state through a dictionary of observables [7]–[9], the nonlinear dynamics become (near-)linear in the lifted space, so a rollout reduces to a sequence of matrix–vector products that are orders of magnitude cheaper

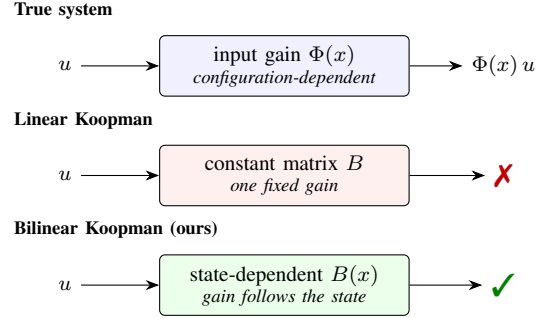


Fig. 1. Why the lift must be bilinear. The command u reaches the tracked output through a *configuration-dependent* gain $\Phi(x)$ (a heading rotation $R(\theta)$ for a vehicle, a Jacobian $J(q)$ for an arm), so the input’s effect is $\Phi(x)u$. A *linear* Koopman lift captures the input-free motion but must apply a single *constant* input matrix, so it cannot reproduce the state-dependent gain (✗). A *bilinear* lift instead lets the input gain $B(x)$ follow the state and matches $\Phi(x)$ (✓), at the same rollout cost; this is made precise in Prop. 1.

than integrating the original dynamics, are trivially parallelized, and, where no analytical model is available, identified directly from data [10], [11]. Recent work couples this idea with MPPI and reports substantial speedups using a learned *linear* Koopman model [12]. Throughout, we assume the analytical dynamics are unknown or too costly to integrate at the sampling rate (the standard model-free, real-time setting), so that a true-dynamics rollout is available only as an *oracle* reference, not as a deployable controller.

The choice of a *linear* lift, however, carries a structural cost that has gone unexamined. A linear Koopman model lets the input act through a single, fixed map, the same regardless of the robot’s configuration. Yet for a large and important class of robots the input does not act this way. A wheeled vehicle’s speed command moves it along its current heading; a manipulator’s joint-velocity command moves the end-effector through its configuration-dependent Jacobian; a legged or whole-body platform drives its task frame through a contact- and posture-dependent map. The pattern is general: across a broad class of robots the input reaches the tracked output through a *configuration-dependent* gain that no constant input map can reproduce (Fig. 1). Neither gain is fixed; each rotates or reshapes with the state, so the linear lift simply averages it away. The *bilinear* Koopman model, the provably correct minimal class for control-affine systems [13], captures precisely this coupling by letting the input multiply the lifted state, and has itself proven effective for control [14]–[16], but only within model predictive control, where the resulting optimization is no longer convex. Linear-predictor Koopman MPC keeps the program convex by construction [17], [18]; sampling-based MPPI carries no such requirement, so the

linear restriction is unnecessary.

This is a structural limitation, not a tuning issue: a constant input matrix *must* fail wherever the gain varies with configuration, and we formalize it as Prop. 1. Because sampling-based control imposes no convexity, we can simply drop the restriction and make the bilinear model the default rollout, at the same matrix–vector regime as the linear one. We further make the controller model-error-aware by injecting Koopman’s proportional error bound [18], [19] as an online error-tube cost (a sampling-based counterpart to robust MPC [20]), and provide a free-energy practical-stability guarantee whose ultimate bound vanishes with the model error. A first- versus second-order criterion then draws the exact boundary of the claim, marking the narrow second-order regime in which a linear lift remains sufficient; it is a scope tool, not the headline. Our contributions are:

- **A representational limitation of linear Koopman lifts.** We identify why linear Koopman models fail for a broad class of robots: a constant input matrix cannot encode a configuration-dependent input gain, so on first-order coupling the linear lift forfeits the input channel rather than merely degrading; this is formalized as Prop. 1 (Sec. III).
- **Bilinear is the right default, at no extra cost.** Because sampling imposes no convexity, the provably correct bilinear class removes this limitation at the same matrix–vector cost as a linear rollout, turning structural failure into success (Sec. III, IV).
- **Error-aware and stable by construction.** We inject the Koopman proportional error bound as an online error-tube cost and establish a free-energy practical-stability guarantee whose ultimate bound vanishes with the model error (Sec. III).
- **The default’s boundary, exactly drawn.** A first- versus second-order geometric criterion closes the precise condition under which a linear lift is *exempted*, demoting the linear case from a competing option to a delimited corner (Sec. III, V).

II. PROBLEM STATEMENT AND BACKGROUND

System and objective. We consider a control-affine system $\dot{x} = f(x) + G(x)u$, $x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$, discretized as

$$x_{k+1} = F(x_k, u_k), \quad (1)$$

known only through input–state data $\{(x_k, u_k, x_{k+1})\}$. Over a horizon T , the controller seeks the input sequence $U = (u_0, \dots, u_{T-1})$ that minimizes the expected cost

$$J(U) = \mathbb{E} \left[\phi(x_T) + \sum_{k=0}^{T-1} q(x_k, u_k) \right]. \quad (2)$$

Information-theoretic MPPI. MPPI draws K input sequences $V^{(k)} = U + \mathcal{E}^{(k)}$ with $\varepsilon_t^{(k)} \sim \mathcal{N}(0, \Sigma)$, rolls each through the dynamics to obtain a trajectory cost $S(V^{(k)})$, and updates each input by the cost-weighted average of the sampled perturbations,

$$u_t \leftarrow u_t + \frac{\sum_k w^{(k)} \varepsilon_t^{(k)}}{\sum_k w^{(k)}}, \quad (3)$$

$$w^{(k)} = \exp \left(-\frac{1}{\lambda} (S(V^{(k)}) - \rho) \right),$$

with temperature $\lambda > 0$ and $\rho = \min_k S(V^{(k)})$. The information-theoretic derivation places no convexity, differentiability, or affine-in-input requirement on F [3]; the only step that scales with the sample count is the per-sample *rollout* used to evaluate $S(V^{(k)})$.

Koopman lifting and identification. Koopman methods lift the state through a dictionary $\Psi : \mathbb{R}^n \rightarrow \mathbb{R}^N$ with $\Psi(0) = 0$ and a left inverse C (so $x = C\Psi(x)$), and model the evolution of the lifted state $z = \Psi(x)$. The *linear* and *bilinear* predictors are

$$(\text{linear}) \quad z^+ \approx Az + Bu, \quad (4)$$

$$(\text{bilinear}) \quad z^+ \approx Az + B_0u + \sum_{i=1}^m u_i (B_i z). \quad (5)$$

Because the input enters the lifted dynamics through state-dependent vector fields, a finite *linear* predictor is generically inexact, whereas the *bilinear* form (5) is the provably correct minimal class [13]. Both are identified by (regularized) least squares, using extended dynamic mode decomposition with control [9], [10] for a fixed dictionary or jointly with a learned deep dictionary [11], so that (5) simply augments the regressor with the products $u_i z$. Writing $M(u) := A + \sum_i u_i B_i$, one bilinear step is $z^+ = M(u)z + B_0u$: an input-dependent matrix, but still $m+1$ matrix–vector products, the same rollout regime as the linear model (4).

Proportional error bound. Let $r(x, u)$ be the one-step residual of (5). Certified Koopman identification [18], [19] provides a *proportional* error bound

$$\|r(x, u)\| \leq c_x \|\Psi(x)\| + c_u \|u\|, \quad (6)$$

whose constants c_x, c_u are estimated from data and *vanish at the equilibrium*. This bound underlies the closed-loop guarantees of Koopman control; in Sec. III we propagate it into the sampling cost.

III. METHOD: BILINEAR KOOPMAN MPPI

Our controller is standard information-theoretic MPPI (3), but every rollout is performed in the lifted space by the *bilinear* surrogate (5). At each control step we lift the measured state, roll K sampled input sequences through $\hat{z}^+ = M(v)\hat{z} + B_0v$, score them with the task cost plus an error-tube penalty, and apply the softmax-weighted update (Algorithm 1). Each rollout step is one $N \times N$ product for A , m for the $\{B_i\}$, and the affine term B_0u —an $m+1$ factor over the single product of a linear model (4). For the small input dimensions typical in robotics this is a modest constant, and the K samples and m products fuse into dense batched matrix multiplications on a GPU, so the wall-clock cost is set by the lifted dimension N , not by the bilinear structure.

Training. Identifying (5) by one-step regression yields poor multi-step rollouts; we instead minimize a *multi-step* loss over length- H windows,

$$\mathcal{L} = \sum_{t=1}^H \|C(\hat{z}_t - z_t)\|^2 + \gamma \|\hat{z}_t - \Psi(x_t)\|^2, \quad (7)$$

where $\hat{z}_t$ is rolled from $\hat{z}_0 = \Psi(x_0)$ through (5) and the second term enforces latent consistency. Two choices matter in practice: we initialize the bilinear matrices B_i to zero, so

Algorithm 1 Bilinear Koopman MPPI (one control step)

Require: state x ; warm start U ; model $A, B_0, \{B_i\}_{i=1}^m, C$;
 bound c_x, c_u ; weights λ, β

- 1: $z_0 \leftarrow \Psi(x)$
- 2: **for** $k = 1, \dots, K$ **in parallel do**
- 3: $V^{(k)} \leftarrow U + \mathcal{E}^{(k)}, \quad \varepsilon_t^{(k)} \sim \mathcal{N}(0, \Sigma)$
- 4: $\hat{z} \leftarrow z_0, \quad e \leftarrow 0, \quad S^{(k)} \leftarrow 0$
- 5: **for** $t = 0, \dots, T - 1$ **do**
- 6: $\hat{z} \leftarrow M(v_t^{(k)}) \hat{z} + B_0 v_t^{(k)}$
- 7: $e \leftarrow (\|M(v_t^{(k)})\| + c_x) e + c_x \|\hat{z}\| + c_u \|v_t^{(k)}\|$
- 8: $S^{(k)} \leftarrow S^{(k)} + q(C\hat{z}, v_t^{(k)}) + \beta e$
- 9: **end for**
- 10: $S^{(k)} \leftarrow S^{(k)} + \phi(C\hat{z})$
- 11: **end for**
- 12: $w^{(k)} \leftarrow \exp(-\frac{1}{\lambda}(S^{(k)} - \min_j S^{(j)}))$
- 13: $U \leftarrow U + (\sum_k w^{(k)} \mathcal{E}^{(k)}) / \sum_k w^{(k)}$
- 14: **return** u_0 ; shift U for the next step

training starts from the linear model (4) and can only improve on it; and we draw coherent control sequences that match the deployment distribution rather than white noise.

Why bilinear is the right default. The bilinear term is what preserves the input channel a linear lift discards, and how much that matters is set by how the input enters the quantity the controller tracks. Let $y = h(x) = Cz$ be that output. To first order in the step, the control-affine dynamics give

$$y_{k+1} - y_k = \underbrace{\nabla h(x_k) f(x_k) \Delta t}_{d(x_k)} + \underbrace{\nabla h(x_k) G(x_k) \Delta t}_{\Phi(x_k)} u_k + O(\Delta t^2), \quad (8)$$

so the input enters y through the state-dependent gain $\Phi(x)$.

Proposition 1 (A constant input matrix cannot encode a state-dependent gain). *Let the tracked output $y = Cz$ evolve, to first order in the step, as $y_{k+1} = y_k + \Phi(x_k) u_k + d(x_k)$ with input gain $\Phi(x)$. A linear Koopman model (4) reproduces this input response for all (x, u) only if Φ is constant; if Φ varies with the configuration, the linear model's output error is of the order of the variation of Φ over the operating set, whereas the bilinear model (5) reproduces $\Phi(x)u$ exactly whenever the dictionary spans the entries of Φ .*

Proof. For (4), $\partial y_{k+1} / \partial u = CB$ is constant, so matching $\Phi(x)$ for all x forces $\Phi \equiv CB$. For (5), $\partial y_{k+1} / \partial u_i = C(B_0 e_i + B_i z)$ is affine in the lifted state, so $\Phi(x)u$ is reproduced once each entry of $\Phi(x)$ lies in $\text{span } \Psi$. $\square$

Remark 1 (first- vs. second-order coupling). The criterion uses the *first-order* (kinematic) increment of y . A wheeled vehicle (output y the position, gain $\Phi(x)$ the heading rotation) and a manipulator (output the end-effector, gain the Jacobian $J(q)$) fall here, so linear Koopman must fail. When the input acts at *second* order, through acceleration (a quadrotor's thrust), $\Phi(x)u$ reaches y only after a further integration, the constant-matrix error is $O(\Delta t^2)$ smaller, and a linear lift is not merely sufficient but preferable. The first-order side is tested

in Sec. IV; the second-order side, where the linear default is safe, in Sec. V.

Examples. For a unicycle (output the planar position) $\Phi(x) \propto (\cos \theta, \sin \theta)$ rotates with the heading; for a manipulator (output the end-effector, joint-velocity input) $\Phi(q) = J(q) \Delta t$ is the Jacobian. A lift spanning these entries reproduces Φ exactly, whereas a constant matrix is correct at one configuration only; as Φ sweeps a wide range its best constant surrogate sits near the average (near zero for the circle-covering unicycle), so the linear model fails outright.

Error-tube cost. The surrogate is inexact, so we discount samples whose *certified* error is large. Propagating the proportional bound (6) along a rollout gives the scalar tube of Algorithm 1 (line 7), with $e_0 = 0$,

$$e_{t+1} = (\|M(v_t)\| + c_x) e_t + c_x \|\hat{z}_t\| + c_u \|v_t\|, \quad (9)$$

and we add $\beta \sum_t e_t$ to each sample cost. Because (6) vanishes at the equilibrium, this penalty regularizes the search toward the certified region; $\beta \rightarrow 0$ recovers plain bilinear MPPI, and replacing the expectation by a conditional value-at-risk yields a risk-averse variant.

Proposition 2 (Tube bound). *Along any shared input sequence, the tube (9) satisfies $\|z_t - \hat{z}_t\| \leq e_t$ for all t , and hence $\|x_t - \hat{x}_t\| \leq \|C\| e_t$.*

Proof. By induction: $e_0 = 0$ matches $z_0 = \hat{z}_0$. Assuming $\|z_t - \hat{z}_t\| \leq e_t$, subtract the true and nominal updates, take norms, and bound the residual by (6) using $\|z_t\| \leq \|\hat{z}_t\| + e_t$; all coefficients are nonnegative, so the bound propagates to $t+1$. $\square$

Stability. Let $\mathcal{F}_\bullet(x) = -\lambda \log \mathbb{E}_{V \sim p}[\exp(-S_\bullet(V; x)/\lambda)]$ be the free energy of the real ($\bullet=\text{real}$) and surrogate ($\bullet=\text{nom}$) costs.

Theorem 1 (Free-energy practical stability). *If the surrogate is cost-controllable and the tracked cost is Lipschitz, then the model-induced free-energy gap obeys $|\mathcal{F}_{\text{real}} - \mathcal{F}_{\text{nom}}| \leq \Delta S_{\text{max}}$, the tube cost of Prop. 2; consequently the real closed loop is practically asymptotically stable with ultimate bound $R = O(c_x, c_u, \lambda, K^{-1/2})$, which vanishes as the model and sampling errors vanish.*

Proof sketch. The gap bound follows from $|S_{\text{real}} - S_{\text{nom}}| \leq \Delta S_{\text{max}}$ (Prop. 2 and cost-Lipschitzness) and monotonicity of $\log \sum \exp$; combined with the relaxed-dynamic-programming decrease of the surrogate value [17], [18] it gives the ultimate bound. Only cost-Lipschitzness is needed, not smoothness of the value function. The full proof and a weighted-norm refinement of (9) (which requires SafEDMD-grade c_x) are in the extended version. $\square$

IV. EXPERIMENTS

Setup. The three controllers share a deep Koopman lifting (identical network width, training data, and multi-step loss) and differ only in the lifted model: *Ours* uses the bilinear form (5) and *MPPI-DK* [12] the linear form (4). For a fair comparison the two learned controllers share the horizon, sample count, lifting dimension, and training budget of Table I;

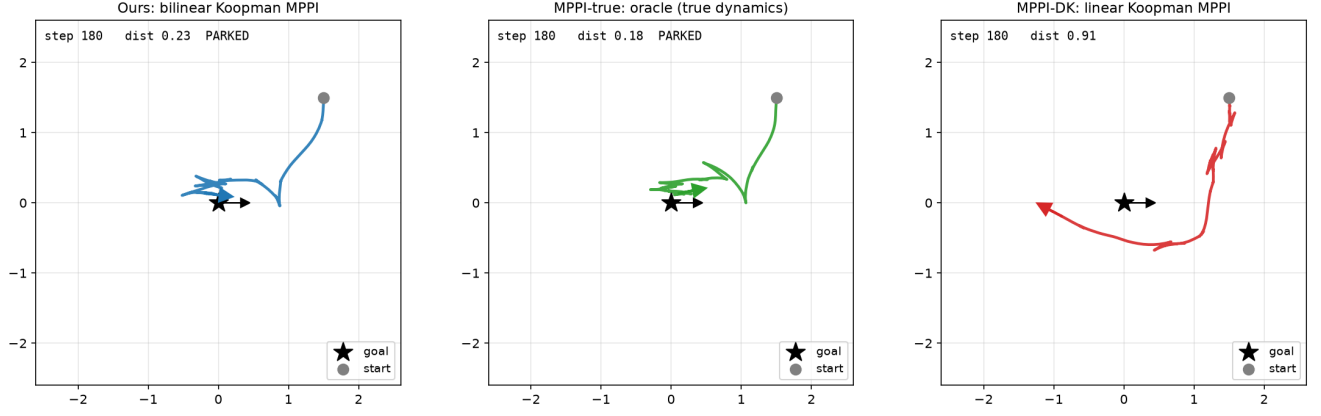


Fig. 2. Nonholonomic unicycle parking (start $\bullet$, goal $\star$; deep Koopman, representative run). Ours (bilinear, left) reaches the goal pose, matching the true-dynamics oracle (centre); MPPI-DK (linear, right) drives off and never converges, since a constant input matrix cannot represent the heading-dependent coupling $v(\cos \theta, \sin \theta)$ (parking rates over 20 trials: 95/85/0%, Sec. IV).

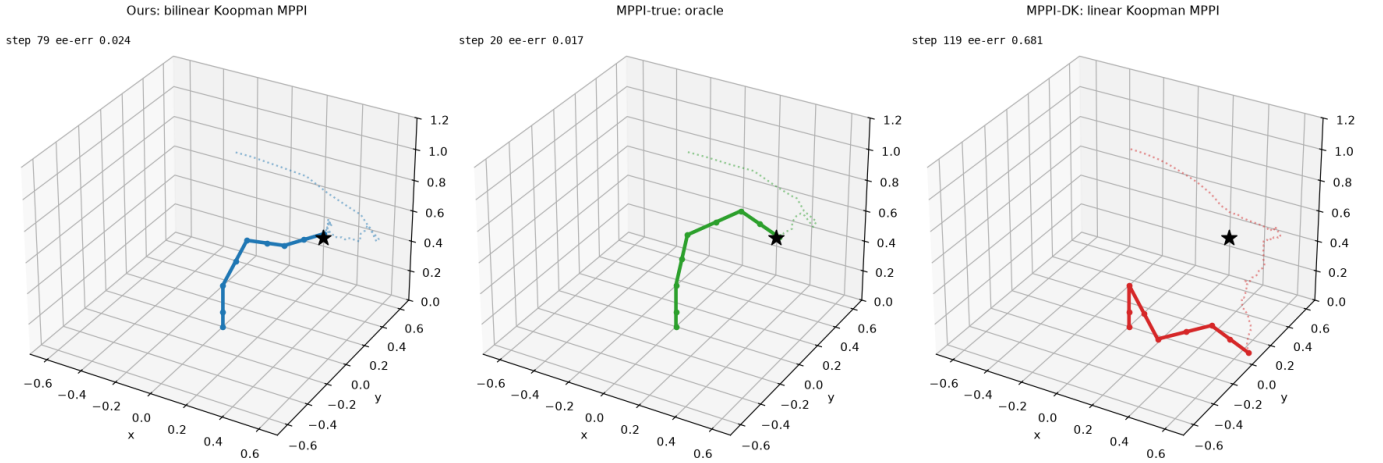


Fig. 3. 7-DOF manipulator end-effector reach (goal $\star$). Ours (bilinear, left) and the oracle (centre) reach the target; MPPI-DK (linear, right) stalls far away, as a constant input matrix cannot represent the configuration-dependent Jacobian $J(q)$.

TABLE I
KEY PER-SYSTEM HYPERPARAMETERS.

System	N	K	T	Δt	λ
Unicycle	8	2048	40	0.05	0.5
7-DOF arm	20	800	15	0.05	0.4
Pendulum	11	1536	50	0.05	0.8
Quadrotor	24	1200	20	0.02	0.6

N , lifted dimension; K , samples; T , horizon; Δt , step size; λ , temperature.

where a setting is method-specific we adopt MPPI-DK’s own reported choice (for instance the pendulum dictionary width), so the reported gaps reflect the model class rather than baseline under-tuning. *Oracle* MPPI, rolling out the true dynamics, is a model-fidelity reference, not a performance ceiling: sampling-based MPPI is suboptimal even with the true model. All systems are simulated in MuJoCo [21]. Key hyperparameters are listed in Table I; full settings and the experiment videos are on the project page: https://koopman_mppi_anonymous.github.io.

A. First-order coupling: where linearizing forfeits the input channel

Nonholonomic vehicle. A unicycle must park its pose (x, y, θ) at a goal; the forward-speed input enters the position through the heading, a configuration-dependent rotation (Prop. 1). Because the heading sweeps the full circle along a trajectory, the best constant surrogate for the input gain $(\cos \theta, \sin \theta)$ sits near its average, which is essentially zero, so the linear lift retains almost no usable input channel. The consequence is stark in open loop: rolled out over the horizon, the bilinear model predicts the position $37\text{--}48\times$ more accurately than the linear one. In closed loop (Fig. 2), ours parks reliably: across five training seeds and four start-goal pairs (20 trials), it reaches the goal pose in 95% of trials, matching the true-dynamics oracle (85%), whereas MPPI-DK parks in 0%, driving off along its initial heading and ending 3.0 ± 1.9 m away.

7-DOF manipulator. A redundant arm must drive its end-effector to a Cartesian target under joint-velocity commands, which enter the end-effector through the configuration-

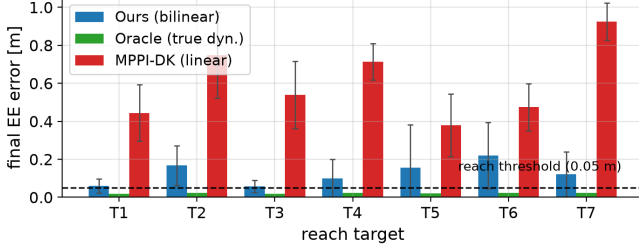


Fig. 4. Final end-effector error over seven targets (mean $\pm$ s.d. across five training seeds). The bilinear model and the oracle stay several times closer to every target than the linear model; MPPI-DK (linear) reaches none on any seed, while the bilinear model lands at oracle accuracy on the targets it solves, with seed-dependent reliability.

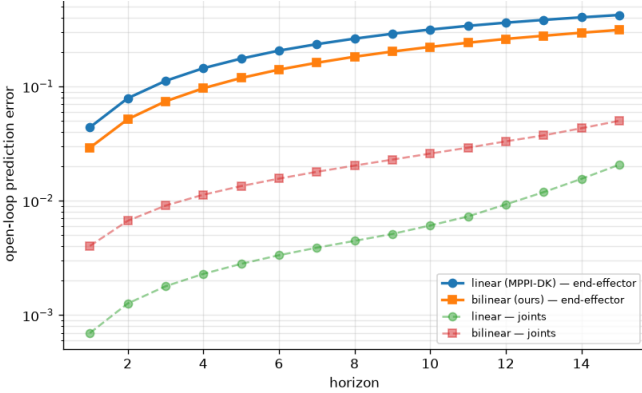


Fig. 5. 7-DOF arm open-loop prediction error vs. horizon. On the end-effector the bilinear edge is modest ($\approx 1.5\times$), because the Jacobian $J(q)$ has a non-zero linearizable mean; receding-horizon control nonetheless amplifies this structural deficit into the closed-loop gap of Fig. 4.

dependent Jacobian $J(q)$. Over seven Cartesian targets and five training seeds (Figs. 3 and 4), the linear model reaches *none* on any seed, ending 0.62 ± 0.26 m from the target: the structural failure of Prop. 1 once more. The bilinear model is about five times closer (mean final error 0.13 m) and lands at oracle accuracy (0.03 m) on the targets it solves. Reliability is lower than on the unicycle, however, for an understood reason: the bilinear model carries one input matrix per channel, so the $m=7$ lift is harder to optimize and more initialization-sensitive—the mechanism behind our zero-initialization and the naive bilinear quadrotor’s instability (Sec. IV-D). The per-seed reach count thus varies (mean $\sim 3/7$, best $6/7$), though the linear model never reaches at all, and reliable optimization of the high-DOF bilinear lift is left open (Sec. V). The open-loop prediction gap is itself modest—about $1.5\times$ on the end-effector (Fig. 5), since $J(q)$ has a non-zero linearizable mean unlike the sign-flipping unicycle gain—so the win is one of structural representability amplified by control, not of prediction-ratio magnitude.

B. Efficiency even where both succeed

On torque-limited pendulum swing-up the input couples to the tracked angle only weakly, and, consistent with MPPI-DK, all three controllers swing up and balance. The informative

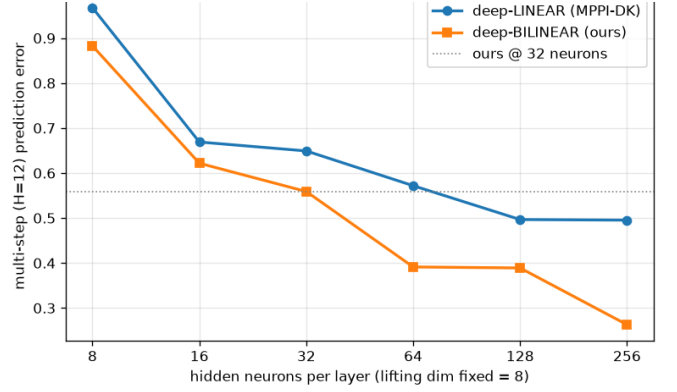


Fig. 6. Pendulum: multi-step ($H=12$) open-loop prediction error vs. dictionary width, at fixed lifted dimension. The bilinear model (ours) reaches a target accuracy with $\approx 4\times$ fewer neurons; the linear model (MPPI-DK) saturates and never reaches the bilinear accuracy. Both controllers swing up.

axis here is not success but model capacity. Fixing the lifted dimension and sweeping the width of the deep dictionary (Fig. 6), the bilinear model reaches a given multi-step accuracy with about $4\times$ fewer neurons: ours at 32 neurons matches the linear model at 128, and the linear model saturates near a prediction error of 0.50 without ever reaching the accuracy (0.26) that ours attains at the 256 neurons MPPI-DK report using. Here bilinearity buys *efficiency* rather than feasibility—the same expressiveness at a fraction of the parameters.

C. Error tube and rollout timing

Tube validity, and the certified-bound caveat. Theorem 1 and the error tube stand on *certified* constants c_x, c_u , which the deep dictionaries used here do not yet supply; we therefore fit the constants by least squares on the deep model’s one-step residuals and treat them as an empirical proxy for the certified bound. Under this proxy the tube is sound: propagated along held-out rollouts of the *deep bilinear* unicycle model, it contains the true lifted error at every step with zero violations over 82,000 rollout steps (Fig. 7), the only out-of-sample violations occurring once a trajectory leaves the region where the bound was fit (6). It is, however, conservative: plain least squares yields $\|A\|_2 \approx 1.02$ and $c_x \approx 3.5 \times 10^{-3}$, so the Euclidean tube compounds to $\sim 10^3\times$ the true error over a 40-step horizon—valid as a certificate of containment, but loose as a bound. The looseness is structural rather than a tuning artefact: the kinematic lift is an integrator ($\|A\|_2 \approx 1$), so even with the small c_x above, a contractive tube (and with it the ultimate bound of Theorem 1) would require a Schur-stable lifted model that plain least squares does not provide, and forcing stability by eigenvalue clipping inflates c_x back in proportion to the clipping margin. Certified, stability-constrained identification [18], [19] is therefore the principal next step (Sec. V).

Rollout timing. The lifted rollout is also why a learned model is preferable to the true-dynamics oracle even in simulation. For $K=1024$ samples over a horizon of $T=30$, one bilinear-Koopman rollout costs 3.5 ms per control step, against 11 ms for a hand-batched integration of the true dynamics ($3\times$)

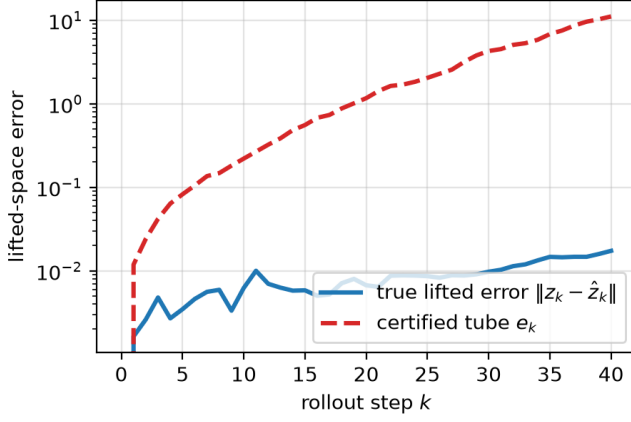


Fig. 7. Error tube e_k (Prop. 2) vs. the true lifted error on a held-out rollout of the *deep bilinear* unicycle model, with c_x, c_u fitted to the model’s residuals as an empirical proxy for certified constants. The tube contains the error at every step (zero violations over 82,000 steps), but with $\|A\|_2 \approx 1.02$ it is conservative; SafEDMD-grade identification would tighten it.

TABLE II
SUMMARY ACROSS SYSTEMS.

System	Coupling	Ours / MPPI-DK	Remark
Unicycle	1st (heading)	park 95%/0%	37–48× pred.
7-DOF arm	1st (Jacobian)	0.13/0.62 m	linear 0/7; high var.
Pendulum	weak	succeed / succeed	4× fewer neurons
Quadrotor	2nd (thrust)	fly / fly	parity

“Coupling” is the order at which the input enters the tracked output; “Ours / MPPI-DK” reports parking rate over 20 trials (unicycle) and mean final error over five seeds (arm). Bilinear is the default; the second-order quadrotor (Sec. V) is the single exception, where a linear lift suffices.

and 72 ms for a physics-engine rollout (20×): the same control rate at a fraction of the compute, the $m+1$ matrix–vector factor over a linear model being negligible against this gap.

D. Ablations and design choices

Three choices proved decisive and double as ablations. *Zero-initialized bilinear terms.* Initializing $\{B_i\}$ to zero makes training start from the linear model (4) and improve monotonically; random initialization instead yields rollout-unstable models—the failure mode behind a naive bilinear quadrotor, whose four input matrices make an unconstrained fit harder to roll out than the linear one. *Multi-step training.* One-step regression of either model gives poor long-horizon rollouts that diverge well before the planning horizon; the H -step loss (7), applied identically to both methods, is what makes either model usable inside MPPI. *The error-tube weight β .* Setting $\beta=0$ recovers plain bilinear MPPI and is best for nominal regulation, where the surrogate is accurate; $\beta>0$ trades a little nominal performance to keep the search inside the certified region, and pays off precisely when the model is least trustworthy—on a weakly identified system or near constraints—consistent with its role as a conservatism knob rather than a performance term.

E. Toward hardware validation

[figure placeholder]

Differential-drive ground robot parking under bilinear vs. linear Koopman MPPI, with motion-capture ground truth. Hardware photograph and overlaid closed-loop trajectories to be added once the study is run.

Fig. 8. [Planned] Real-robot parking. Anticipated outcome: the bilinear-model controller (ours) reaches the goal pose, while the linear-model controller (MPPI-DK) veers off, reproducing the simulation result of Fig. 2 on hardware.

[Planned — not yet run.] The simulations above isolate the mechanism in a noise-free setting; a hardware study is in preparation to confirm it survives real sensing, actuation, and identification noise. The primary platform is a differential-drive ground robot, whose forward-speed and yaw-rate commands enter its planar position through the heading—the canonical first-order, configuration-dependent coupling of Prop. 1, and thus the sharpest real-world test of the criterion. A bilinear and a linear deep Koopman model are identified from the same on-board odometry logs (several minutes of teleoperated and randomized driving) with no analytical model supplied to either controller. Both controllers then run a goal-reaching and parking task at the platform’s native control rate, with pose ground truth from an overhead motion-capture system. We anticipate the simulation gap to persist on hardware: the bilinear-model controller should park within a few centimetres of the goal across a battery of start–goal pairs, whereas the linear-model controller, unable to represent the heading rotation, should systematically veer off and stall—the closed-loop reflection of the 37–48× open-loop prediction gap of Sec. IV. A second study on a velocity-controlled 6-DOF manipulator (Jacobian coupling) is planned to reproduce the reach-versus-fail outcome of Fig. 4 on real hardware. Final measurements, success rates, and videos will be posted to the project page.

V. DISCUSSION AND CONCLUSION

This paper makes a single claim: for sampling-based predictive control of control-affine systems, the bilinear Koopman model is the right default rollout. Sampling removes the only reason the field has kept the lift linear (the convexity that Koopman MPC requires and MPPI does not), so the provably correct bilinear class costs nothing extra to roll out, and the experiments show that using it rather than a linear surrogate is what separates reaching the goal from structural failure on first-order-coupled robots. Keeping the lift linear is therefore not a neutral simplification but a forfeiture of the input channel; the accompanying error-tube cost and free-energy guarantee make the bilinear rollout error-aware and practically stable.

The boundary of the default. The claim is not that bilinearity is universally superior, and a quadrotor marks the precise edge where it is not needed. When the input reaches the tracked output only at *second* order (thrust drives

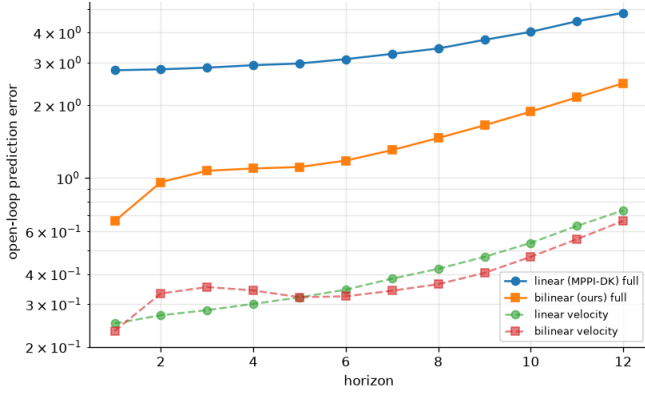


Fig. 9. The boundary case. Quadrotor open-loop prediction error vs. horizon: the bilinear model is 2–4 $\times$ better on the *full* state (it captures the thrust–attitude coupling), but on the control-relevant *velocity* the linear and bilinear curves coincide. Second-order, acceleration-mediated coupling is the delimited regime where a linear lift suffices—indeed is preferable.

position through acceleration), the state-dependence of the input map is integrated once more and diluted to $O(\Delta t^2)$, so a constant input matrix is no longer a forfeiture. The split is visible in prediction (Fig. 9): the bilinear model is 2–4 $\times$ more accurate on the *full* state, capturing the thrust–attitude coupling, yet on the control-relevant *velocity* the two models coincide and both controllers fly to the goal. Here a linear lift is in fact *preferable*: fewer parameters and an easier fit, with the bilinear model’s extra input matrices offering no return and, left untreated, even hurting trainability. We read this not as a failure mode but as a cleanly closed exemption: the first- versus second-order criterion (Prop. 1, Remark 1) tells a practitioner exactly when the linear default is safe, and that corner is narrow and characterizable.

Relation to prior work and scope. The result complements rather than competes with linear Koopman control: linear-predictor Koopman MPC [17], [18] and MPPI-DK [12] remain well matched to the second-order and weakly coupled regime above, and the contribution here is to lift the linear restriction wherever first-order coupling makes it costly. The same criterion bounds our own scope: it presumes a tracked output that couples to the input through smooth first-order kinematics, so contact-rich, hybrid dynamics such as legged locomotion, which interleave configuration-dependent coupling with switching contacts that no smooth Koopman lift captures cleanly, are left to future work.

Two limitations remain. First, optimizing the bilinear model grows harder with the input dimension (Sec. IV): high-DOF lifts train less reliably than the convex linear fit, so closed-loop success, oracle-accurate when achieved, varies across seeds, and stability-constrained identification is the natural remedy. Second, our stability guarantee presumes a certified error bound that the deep dictionary does not yet furnish, so the fitted constants of Sec. IV stand only as an empirical proxy; supplying genuine certificates through certified (kernel- or SafEDMD-based) identification, which would also yield the small constants a contractive tube requires, together with the hardware studies, are the immediate next steps. The takeaway is a default with a known boundary: in sampling-

based Koopman control, roll out the bilinear model unless the input reaches the tracked output only at second order.

REFERENCES

- [1] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Aggressive driving with model predictive path integral control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2016, pp. 1433–1440.
- [2] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic MPC for model-based reinforcement learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 1714–1721.
- [3] G. Williams, A. Aldrich, and E. A. Theodorou, “Model predictive path integral control: From theory to parallel computation,” *J. Guid. Control Dyn.*, vol. 40, no. 2, pp. 344–357, 2017.
- [4] H. J. Kappen, “Path integrals and symmetry breaking for optimal control theory,” *J. Stat. Mech.: Theory Exp.*, p. P11011, 2005.
- [5] E. A. Theodorou, J. Buchli, and S. Schaal, “A generalized path integral control approach to reinforcement learning,” *J. Mach. Learn. Res.*, vol. 11, pp. 3137–3181, 2010.
- [6] L. Park, K. Jang, and S. Kim, “A systematic survey of model predictive path integral control: From stochastic theory to real-time robotics,” 2026, under review, ISA Transactions.
- [7] B. O. Koopman, “Hamiltonian systems and transformation in Hilbert space,” *Proc. Natl. Acad. Sci.*, vol. 17, no. 5, pp. 315–318, 1931.
- [8] I. Mezić, “Spectral properties of dynamical systems, model reduction and decompositions,” *Nonlinear Dyn.*, vol. 41, pp. 309–325, 2005.
- [9] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, “A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition,” *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [10] J. L. Proctor, S. L. Brunton, and J. N. Kutz, “Dynamic mode decomposition with control,” *SIAM J. Appl. Dyn. Syst.*, vol. 15, no. 1, pp. 142–161, 2016.
- [11] B. Lusch, J. N. Kutz, and S. L. Brunton, “Deep learning for universal linear embeddings of nonlinear dynamics,” *Nat. Commun.*, vol. 9, p. 4950, 2018.
- [12] W. Hao, Y. Fang, Z. Lu, and S. Mou, “Accelerating sampling-based control via learned linear Koopman dynamics,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2026, arXiv:2603.05385.
- [13] L. C. Iacob, R. Tóth, and M. Schoukens, “Koopman form of nonlinear systems with inputs,” *Automatica*, vol. 162, p. 111525, 2024.
- [14] D. Bruder, X. Fu, and R. Vasudevan, “Advantages of bilinear Koopman realizations for the modeling and control of systems with unknown dynamics,” *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4369–4376, 2021.
- [15] C. Folkestad and J. W. Burdick, “Koopman NMPC: Koopman-based learning and nonlinear model predictive control of control-affine systems,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021, pp. 7350–7356.
- [16] G. Mamakoukas, M. L. Castano, X. Tan, and T. D. Murphey, “Local Koopman operators for data-driven control of robotic systems,” in *Robotics: Science and Systems (RSS)*, 2019.
- [17] M. Korda and I. Mezić, “Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control,” *Automatica*, vol. 93, pp. 149–160, 2018.
- [18] R. Strässer, M. Schaller, K. Worthmann, J. Berberich, and F. Allgöwer, “An overview of Koopman-based control: From error bounds to closed-loop guarantees,” *Annual Reviews in Control*, vol. 61, p. 101035, 2026.
- [19] —, “SafEDMD: A certified learning architecture tailored to data-driven control of nonlinear dynamical systems,” arXiv:2411.10359, 2024.
- [20] G. Williams, B. Goldfain, P. Drews, K. Saigol, J. M. Rehg, and E. A. Theodorou, “Robust sampling based model predictive control with sparse objective information,” in *Robotics: Science and Systems (RSS)*, 2018.
- [21] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.